

Threat Modelling and Risk Analysis for Large Language Model (LLM)-Powered Applications

Stephen Burabari Tete, *Teesside University UK*

Abstract—The advent of Large Language Models (LLMs) has revolutionized various applications by providing advanced natural language processing capabilities. However, this innovation introduces new cybersecurity challenges. This paper explores the threat modeling and risk analysis specifically tailored for LLM-powered applications. Focusing on potential attacks like data poisoning, prompt injection, SQL injection, jailbreaking, and compositional injection, we assess their impact on security and propose mitigation strategies. We introduce a framework combining STRIDE and DREAD methodologies for proactive threat identification and risk assessment. Furthermore, we examine the feasibility of an end-to-end threat model through a case study of a custom-built LLM-powered application. This model follows Shostack's Four Question Framework, adjusted for the unique threats LLMs present. Our goal is to propose measures that enhance the security of these powerful AI tools, thwarting attacks, and ensuring the reliability and integrity of LLM-integrated systems.

Index Terms—cybersecurity, threat modelling, risk analysis, large language model, LLM, AI.

I. INTRODUCTION

AS the technological domain continues to embrace the power of artificial intelligence, Large Language Models (LLMs) are at the forefront of this revolution, transforming how we interact with and benefit from machine intelligence. LLMs have demonstrated an uncanny ability to understand and generate human-like text, propelling a wide array of applications to new heights of capability. With this evolution, identifying and mitigating the underlying risks through robust threat modeling and risk analysis is becoming a critical necessity for maintaining the integrity and trustworthiness of LLM-powered systems. Groundbreaking studies and meticulous surveys have laid the foundation for understanding the risks associated with these models, highlighting the dual nature of technology as both an asset and a vulnerability [1].

A. LLM-Powered Applications

LLMs such as GPT-4 and PaLM 2 signify a leap forward in natural language processing, boasting advanced capabilities in understanding context and producing relevant responses [2], [3]. These models have become integral in numerous real-life applications, ranging from augmented search engines to sophisticated virtual assistants. For instance, GPT-4's integration into Microsoft's Bing Search is a testament to its practical utility, while Google's Bard demonstrates the potential of PaLM 2 as a powerful information retrieval tool [1], [4].

Beyond the realm of search and retrieval, LLM-integrated applications are redefining boundaries in other sectors. In

healthcare, LLMs are exploring uncharted territories, from enhancing patient communication through AI chatbots to long-term risk prediction for chronic conditions, offering invaluable insights into preemptive care [5]–[11]. Each application harnesses the strengths of these models, from analyzing prompts to synthesizing responses in intricate tasks like spam detection, translation, and more. By combining instruction prompts guided by human intent with data prompts sourced from external resources, these applications provide tailored and contextually apt solutions for users [2], [12]–[14].

B. The Threats and Risks

Harnessing the transformative power of LLMs does not come without its threats and risks. As the applications grow more complex and integral to our infrastructure, the potential repercussions of security breaches or faulty outputs become increasingly severe. Gleaning insights from comprehensive surveys on machine-generated text, we are reminded that threat models should be meticulously crafted to detect, analyze, and mitigate any form of malicious infiltration or unintended consequence [1].

The integration of LLMs in sensitive sectors like healthcare further underscores the urgency for robust threat modeling. With studies illustrating the promise of LLMs in managing health-related risks, it is evident that ensuring their security is non-negotiable [5]–[11]. Keeping confidential patient data safe while providing accurate predictions and diagnostics requires an in-depth understanding of cyber threats, ranging from data breaches to sophisticated adversarial attacks that could exploit model vulnerabilities.

The management of these threats begins with identifying the scope of potential risks, guided by frameworks that consider the entire lifecycle of the system [1]. From development through to deployment, each phase presents unique challenges that must be addressed proactively. Through the adoption of security development life cycles and consistent security reviews, as Microsoft has demonstrated with the trustworthy computing security development lifecycle, we can significantly reduce the rate of vulnerabilities in software underpinned by LLMs [2], [4].

In this paper, our primary goal is to examine the threats and risk of LLM-powered applications, as well as possible mitigation to prevent or reduce the threats and risks. In particular, we focus on the following research questions (RQ):

- **RQ1: What are the potential attacks against LLM-powered applications and what are their impact on the application's security?** To study this question, we

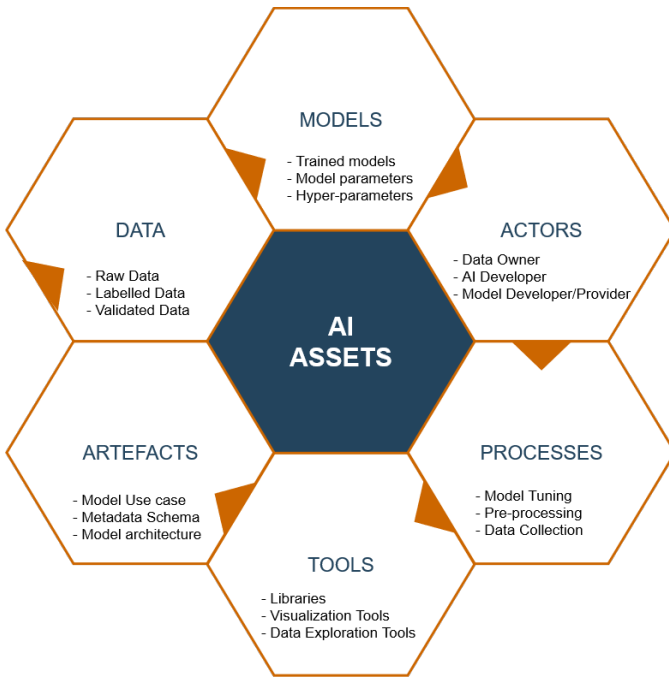


Fig. 1. AI Assets

focused on web applications built upon Flask framework, using OpenAPI's GPT-3.5 and GPT-4 models.

- **RQ2: What defence or mitigation can prevent or reduce the effectiveness of these attacks?** To answer this question, we reviewed previous mitigation provided and validated by several scholars and featured it together.
- **RQ3: Is an end-to-end threat model for LLM-powered application feasible?** To address this question, we took a case study of our custom built LLM-Doctor [15] application, powered by GPT-3.5 and GPT-4 and designed a custom end-to-end threat model.

II. OVERVIEW OF THREAT MODELLING, AND RISK ANALYSIS

A. Threat Modelling (TM)

Threat modelling is a structured approach for identifying and prioritizing potential threats to a system and determining the value that potential mitigations would have in reducing or neutralizing those threats. This process is essential in the proactive identification and management of cybersecurity risks, particularly in complex distributed systems, such as those that harness the capabilities of Large Language Models (LLMs).

The evolution of threat modelling has paralleled advancements in the software development life cycle (SDLC), with various methodologies being developed to suit distinct environments. Initially conceptualized through the CIA triad proposed by the Johnson Space Center in the 1980s, which underlined the importance of confidentiality, integrity, and availability, threat modeling methodologies have expanded to include additional elements such as authentication, non-repudiation, and authorization, as seen in the STRIDE model [16], [17].

One of the seminal works in threat modelling was introduced by Schneider in 1998 with the use of attack trees as a means of graphically representing and analyzing security threats, providing a foundational approach to threat modeling [18]. Today, threat modeling techniques such as STRIDE (an acronym for Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege) provide a systematic and comprehensive method for identifying potential security threats to software applications [19]).

Moreover, the demand for rigorous security measures throughout the SDLC is apparent in the Trustworthy Computing Security Development Lifecycle (SDL) advanced by Microsoft [20], in which software undergoes a final security review by an independent team and demonstrated a significant reduction in the discovery of security vulnerabilities compared to non-SDL-adhered software.

As machine learning (ML) and artificial intelligence (AI) become increasingly integral to numerous applications, the need to model threats specifically to AI-ML systems is more critical than ever [21]; [19]). This is particularly important as we continue moving towards systems with substantial autonomy, requiring a comprehensive life cycle approach to the design and development of AI solutions [22].

B. TM Framework

Given the complexity and novelty of LLM-powered applications, it is of paramount importance to address their security comprehensively. Our proposed framework for threat modeling and risk analysis combines strategies from established methodologies that address both the intricacies of AI systems and existing best practices in security assessment.

STRIDE, with its categorization of threats, serves as the ideal foundational model for threat modeling in LLM-powered applications. It comprehensively covers the multitude of potential vulnerabilities and the rising need to fortify AI-ML systems against increasingly sophisticated threats [19]). The inclusion of elements from Shostack's Four Question Framework enhances the effectiveness of STRIDE, facilitating an improved understanding of security goals, potential attackers, likely attack vectors, and the impact of successful attacks.

C. Risk Analysis

DREAD, an acronym for Damage, Reproducibility, Exploitability, Affected Users, and Discoverability, offers a risk analysis methodology that allows for a more granular evaluation of the severity of potential threats identified during the threat modelling process. The coupling of STRIDE with DREAD aligns with the enhancement of threat modeling outcomes, as evidenced by a study focused on distributed control systems in the oil industry [16]).

Furthermore, the complexity and advanced capabilities of LLMs, evidenced by state-of-the-art natural language generation (NLG) systems, necessitates a threat modeling framework that accounts for new vectors of abuse specific to such systems [23]). Therefore, our proposed framework not only incorporates classic methodologies but also addresses the novel risks posed by the deployment of LLM-powered applications.

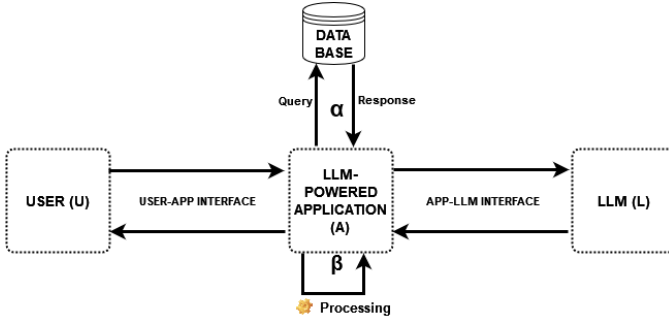


Fig. 2. LLM-Powered application scheme

Dataset name	Size ($\times 10^6$)	Release date	Cryptographic hash?	Data from expired domains	Data buyable for \$10K USD	Downloads per month
LAION-2B-en [57]	2323	2022	χ^1	0.29%	$\geq 0.02\%$	≥ 7
LAION-2B-multi [57]	2266	2022	χ^1	0.55%	$\geq 0.03\%$	≥ 4
LAION-1B-nolang [57]	1272	2022	χ^1	0.37%	$\geq 0.03\%$	≥ 2
COYO-700M [11]	747	2022	χ^3	1.51%	$\geq 0.15\%$	≥ 5
LAION-400M [58]	408	2021	χ	0.71%	$\geq 0.06\%$	≥ 10
Conceptual 12M [16]	12	2021	χ	1.19%	$\geq 0.15\%$	≥ 33
CC-3M [65]	3	2018	χ	1.04%	$\geq 0.11\%$	≥ 29
VGG Face [49]	2.6	2015	χ	3.70%	$\geq 0.23\%$	≥ 3
FaceScrub [46]	0.10	2014	$\checkmark^3$	4.51%	$\geq 0.79\%$	≥ 7
PubFig [34]	0.06	2010	$\checkmark^{3*}$	6.48%	$\geq 0.48\%$	≥ 15

Fig. 3. All recently-published large datasets are vulnerable to split-view poisoning attacks. (Carlini et al., 2023)

Our comprehensive review of cybersecurity risk assessment methods for complex systems underscores the importance of strategic planning throughout the system lifecycle, which is palpable in the integration of threat modeling and risk analysis from the conception phase of LLM-powered applications through to their production and deployment [17]).

By integrating STRIDE and Shostack's Four Question Framework for effective threat identification, and employing DREAD for in-depth risk analysis, the proposed framework intends to elevate the security posture of LLM-powered applications throughout their entire life cycle, thus ensuring robustness against an evolving threat landscape.

III. POTENTIAL ATTACKS ON LLM-POWERED APPS (RQ1)

With the increasing integration of Large Language Models (LLMs) across various domains, an understanding of potential security threats has become paramount. Given their deep interconnection with data and subjective decision-making capabilities, LLM-powered applications are susceptible to a myriad of sophisticated attacks. This section explores a range of potential attacks, leveraging various research works to delineate the nature and feasibility of these adversarial strategies.

A. Data Poisoning Attack

Data poisoning is a pernicious form of attack employing the subtle manipulation of training data. By contaminating the dataset, an adversary can skew the model's behavioral patterns, potentially causing intended misclassification or biased outputs. Carlini et al. (2023) highlight the significant risks associated with training deep learning models on large-scale datasets amassed from the internet [24]. These datasets are vulnerable because they rely on the mutable nature of web content, which can be manipulated to include poisoning instances

intended to skew the model's learned behavior. The research emphasizes how poisoning can be both straightforward and cost-effective, making it a tangible threat to the integrity of machine learning models [25].

Techniques for Data Poisoning

- 1) **Split-View Poisoning** This method takes advantage of the dynamic nature of internet resources. By altering the content after it has been initially indexed, attackers can ensure that what gets included in the dataset later differs from the original. Such discrepancies can mislead annotators and poison the dataset, influencing the bias and the decisions of the trained models [25].
- 2) **Frontrunning Poisoning** Data poisoning can also occur in datasets that snapshot user-generated or crowd-sourced content at regular intervals. By timing malicious changes to appear during these snapshots, attackers can insert poisoning examples without needing to maintain long-term control over the content [25].

Implications of Data Poisoning: These poisoning techniques undermine trust in the data's integrity and raise concerns about the safety and fairness of the resulting models. An adversary can strategically introduce errors leading to selected misclassification or biased model outputs. The ease and affordability of these attacks underscore the urgency to develop robust mitigation strategies.

B. Prompt Injection Attacks

A quintessential vulnerability in LLMs is the exploitation of the prompt mechanism. Attackers craft malicious inputs designed to deceive or misguide the model into producing unintended outcomes. Yi et al. (2023) and Liu et al. (2023) emphasize the severity of this issue, illustrating how indirect prompt injection attacks can present significant challenges to the integrity of LLMs [26], [27]. This vulnerability can cascade severely, affecting LLM-integrated web applications, such as Rodrigo Pedro and colleagues have shown in their work, potentially leading to SQL injection attacks, pivoting from seemingly innocuous prompt manipulations [28].

C. SQL Injection Attacks

The long-standing plague of SQL injection continues to evolve with the advent of LLMs. Attackers leverage crafted prompts or inputs to coerce an LLM into generating or executing harmful SQL code. This type of intrusion can jeopardize data integrity as well as confidentiality. The work by Kindy and Pathan (2013), although predating current LLM complexities, still provides foundational insights into understanding such invasive techniques [29]. Furthermore, Jahanshahi et al. (2020) discuss mitigation strategies, indicating that the robustness against SQL injection attacks remains an ongoing concern and necessitates novel defence mechanisms tailored to LLM environments [30].

D. Jailbreaking Attacks

Jailbreaking [31] attacks directly confront the restrictive measures placed within LLMs, effectively 'breaking out' of

the intended operational parameters. Mehrotra et al. (2023) and Wu et al. (2023) both address the alarming efficiency with which attackers can execute self-adversarial strategies to jailbreak black-box models, including highly advanced versions like GPT-4 [32] [33]. Zhang et al. (2023) and Robey et al. (2023) explore defense mechanisms, emphasizing the need for adaptive and resilient model structures to mitigate such risks [34] [35].

E. Compositional Injection Attacks

This attack vector introduces a complex combination of instructions that embed hidden attacks within an otherwise benign prompt. Jiang et al. (2023) present the 'Prompt Packer' concept, where an adversary deceives an LLM through multi-layered, compositional instructions with nefarious intent [36]. Such compositional injections could bypass conventional safeguard measures and achieve an unauthorized influence on model behavior.

F. Insecure Output Handling

As the number two (2) vulnerability on the OWASP 2023 Top 10 for LLM application, insecure output from LLM can lead to: CRSF, SSRF, and XSS, if the output is exploited successfully. [37]

Attack scenario: An LLM-powered application processes output from the LLM directly, without first validating the output. This enables the attacker to manipulate the system, by executing harmful output from the LLM, which the attacker deliberately caused through crafted prompt.

IV. MITIGATION TO ATTACKS ON LLM-POWERED APPS (RQ2)

A. Mitigation for Data Poisoning

Mitigating Frontrunning Poisoning Randomized Timing: By randomizing when snapshots of data are taken or by extending the time required to snapshot an entire dataset curators make it more difficult for an attacker to predict when to execute the poisoning.

Review and Freeze Period: Instituting a review period where edits to the content are frozen before officially capturing the snapshot allows for the detection and removal of any malicious modifications.

Mitigating Poisoning in General Relying on Consensus: For even broader datasets like Common Crawl, where poisoning prevention is more challenging, one proposed solution is to employ a consensus-based approach. Trust would be conferred on data elements that appear consistently across a range of sources, thus requiring attackers to compromise many instances, paralleling distributed systems' trust mechanisms. **Developing Unique Defenses** Given the variety of data use cases and the complexities involved in understanding how content is used and interpreted by learning algorithms, specific solutions are required. These defenses would need to rely on in-depth knowledge of the data consumption and training process, and would vary based on the dataset and potential attack vectors.

While the [38] outlines potential mitigation strategies, the authors also acknowledge the need for further research on application-specific solutions. By understanding the complexities of content vectorization and conflict resolution in the training process, more effective defenses against data poisoning could be developed.

B. Mitigation for prompt injection

Prompt injection [39], [26] comes with different attack techniques, style and pattern. Indirect, direct injection. Prompt injection attacks present a significant threat to the trustworthiness and security of Language Model (LM) integrated applications. These attacks exploit weaknesses in the processing of prompts by the underlying models, potentially leading to incorrect, unwanted, or even malicious responses. In this section, we explore various mitigation strategies, with a focus on both prevention-based and detection-based defenses, to safeguard these systems.

Prevention-Based Defenses: Prevention is the first line of defense against prompt injection attacks, aiming to safeguard the system before any damage can occur.

- 1) Paraphrasing: Paraphrasing disrupts the structure of injection attacks by altering the sequence of characters and words, making it difficult for injected instructions to remain coherent within the altered data prompt. This technique was initially developed to defend against adversarial prompts [40] but has since been adapted to address prompt injection threats [39], [41].
- 2) Retokenization: Retokenization involves splitting tokens into smaller units using methods like BPEDropout [42]. This approach can obfuscate malicious instructions or data embedded in the data prompt, neutralizing the adversarial content [41].
- 3) Data Prompt Isolation: By isolating the data prompt with delimiters such as three single quotes, XML tags, or random sequences, the LM is forced to treat the data prompt strictly as data, ignoring any additional instructions that are not part of the designated task [43].

C. Mitigation for Insecure Output Handling

Apply the Zero Trust architecture, by treating the LLM as a user whose data must be validated. Validate every output coming from the LLM to the application's backend (classes and functions) before processing, displaying to users or saving to the database. [37]

V. END-TO-END THREAT MODEL FOR LLM-POWERED APPS

Threat modeling is analyzing representations of a system to highlight concerns about security and privacy characteristics [44]. Secure software development requires a 'shift left' — paying attention to security and privacy early in the life cycle. Threat modelling is a very useful activity for achieving this goal, but for a variety of reasons, organizations struggle to introduce it [45].

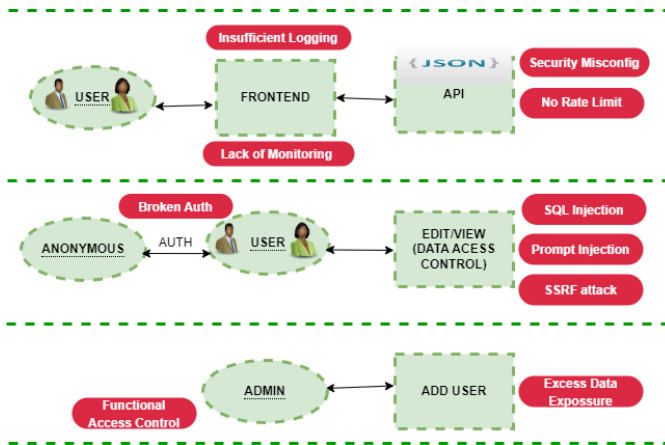


Fig. 4. LLM-Application Threat Model

Using one of the best threat modelling framework [23], [44], [46], we adapt the Shostack's 4 Question Frame for Threat Modelling [46], [47], which presents the following questions:

- 1) What are we working on?
- 2) What can go wrong?
- 3) What are we going to do about it?
- 4) Did we do a good job?

A group of 15 security and privacy researchers [44], band together in order to publish a new manifesto designed to guide organizations on their threat modeling journeys, it's called 'Threat Modelling Manifesto' [44]. Proposed by these researchers are the following values, which are preferred, over others. We have come to value:

- A culture of finding and fixing design issues over checkbox compliance.
- People and collaboration over processes, methodologies, and tools.
- A journey of understanding over a security or privacy snapshot.
- Doing threat modeling over talking about it.
- Continuous refinement over a single delivery.

The manifesto aims to guide organizations in effective threat modeling, emphasizing its broader utility beyond predicting attacks. It addresses challenges faced by companies in implementing coherent models and highlights the need for accessibility beyond the security team. [44], [47], [48]

The researchers argue that a good threat model goes beyond predicting attacks, aiding in identifying unknown vulnerabilities, estimating risk, and planning security scenarios. The manifesto emphasizes the importance of making threat modeling understandable to a wider audience, including executives and developers, challenging the perception of it as a complex process. [44], [48]

The document encourages organizations to avoid common pitfalls in the threat modeling process and advocates for a workplace culture emphasizing problem-solving, collaboration, and continuous model updates. While not prescribing specific methods, the manifesto outlines high-level values and principles for effective threat modeling, cautioning against overly complex models.

Aligned with agile development principles, the manifesto promotes an iterative and cyclical approach to threat modeling, emphasizing its compatibility with the dynamic nature of security needs. It concludes by suggesting that organizations embracing DevSecOps trends are likely to integrate threat modeling into their product development initiatives. The manifesto aims to facilitate the understanding and adoption of threat modeling practices as a continuous and evolving process. [44], [47]

A. Threat Modelling with STRIDE & DREAD

Creating a threat model involves identifying potential threats and vulnerabilities in a system to understand and mitigate risks. Using STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege) and DREAD (Damage, Reproducibility, Exploitability, Affected Users, and Discoverability) which are two commonly used threat modeling frameworks; we create a threat model for our LLM-powered application: Threat Model for LLM-Powered Application

1) **Spoofing (STRIDE):**

Threat: Unauthorized access to the application or API using fake credentials. DREAD: Damage: High (Potential compromise of sensitive information) Reproducibility: High (Could be attempted by malicious actors) Exploitability: Medium (Depends on the strength of authentication mechanisms)

2) **Tampering (STRIDE):**

Threat: Modification of input data to the language model for malicious purposes (e.g., prompt injection). DREAD: Damage: High (Altered or biased model outputs) Reproducibility: Medium (Requires understanding of model input mechanisms) Exploitability: High (Common attack vector)

3) **Repudiation (STRIDE):**

Threat: Denying the execution of certain actions, such as generating specific outputs from the language model. DREAD: Damage: Medium (Could lead to disputes or lack of accountability) Reproducibility: Medium (Depends on the type of repudiation) Exploitability: Low to Medium (May require sophisticated attacks)

4) **Information Disclosure (STRIDE):**

Threat: Unauthorized access to sensitive information generated by the language model. DREAD: Damage: High (Potential exposure of sensitive data) Reproducibility: Medium (Depends on the nature of information disclosure) Exploitability: Medium (Depends on security controls)

5) **Denial of Service (STRIDE):**

Threat: Overloading the API with a large number of requests to disrupt normal functioning. DREAD: Damage: High (Service downtime and impact on user experience) Reproducibility: High (Common attack vector) Exploitability: High (Potential for abuse).

6) **Elevation of Privilege (STRIDE):**

Threat: Gaining unauthorized access to higher privileges within the application or system. DREAD: Damage:

TABLE I
SUMMARY OF SOME PROMPT INJECTION ATTACKS TO LLM-POWERED APPLICATIONS.

Attack Name	Attack Description	Source
Naive Attack	Concatenate target data, injected instruction, and injected data	Online post: [49]–[51]
Escape Characters	Adding special characters like “\n” and “\t”	Arxiv paper: [38]
Context Ignoring	Adding context-switching text to mislead the LLM that the context changes	Workshop paper: [52] Arxiv paper: [53] Online post: [49], [54]
Fake Completion	Adding a response to the target task to mislead the LLM that the target task has completed	Online post: [55]
Combined Attack	Combining Escape Characters, Context Ignoring, and Fake Completion	Arxiv paper: [38]

High (Compromise of critical system components) Reproducibility: Low (May require multiple vulnerabilities) Exploitability: Low (Depends on the existing security controls)

Threat Modelling with Shostack's 4 Question Frame

- **What Are We Working On?**
 - Development of an LLM-powered application for natural language processing tasks.
- **What Can Go Wrong?**
 - Identified threats include Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.
- **What Are We Going to Do About It?**
 - Implemented multi-factor authentication to address Spoofing.
 - Applied input validation and content integrity checks to prevent Tampering.
 - Established robust logging and auditing mechanisms for Repudiation.
 - Encrypted sensitive data and enforced access controls for Information Disclosure.
 - Implemented rate limiting and traffic analysis for Denial of Service.
 - Adopted the principle of least privilege and conducted regular access control reviews for Elevation of Privilege.
- **Did We Do a Good Enough Job?**
 - Continuous threat modeling, security testing, and feedback mechanisms are in place to ensure ongoing improvement.

VI. CONCLUSION

As the technological ecosystem rapidly adopts Large Language Models (LLMs), the need for comprehensive threat modeling and risk analysis becomes imperative to safeguard the applications they empower. This study has extensively examined the threats to LLM-powered applications, highlighting potential attacks such as data poisoning, prompt injection, SQL injection, jailbreaking, and compositional injection attacks. Through an adaptive and holistic threat modeling approach, this paper advocates a robust security framework merging

STRIDE and DREAD methodologies, offering concrete mitigation strategies tailored for these sophisticated AI systems.

The exploration of an end-to-end threat model for a custom-built LLM-Doctor [15] application validates the framework's feasibility and demonstrates its effectiveness in identifying, categorizing, and mitigating threats. By following a methodical approach, this research provides valuable insights and security paradigms that can be implemented across disciplines and industries where LLMs are utilized. However, the dynamic nature of cyber threats and the continuous evolution of LLMs necessitate an ever-vigilant and responsive approach to threat modeling in AI-powered software development.

VII. LIMITATIONS

This paper focuses on threat modelling and risk analysis for LLM-powered applications, in which the LLM is a third party solution, which is accessed through application programming interface (API) only. Other research may explore the use of in-house LLM.

VIII. RECOMMENDATIONS FOR FURTHER RESEARCH

Based on the findings of this study, several recommendations can be made to guide further research in the area of securing LLM-powered applications:

- 1) **Dynamic Threat Modeling:** Develop adaptive threat modeling methodologies that evolve with LLM advancements and emerging attack vectors, ensuring continual alignment with the changing threat landscape.
- 2) **Automated Response Mechanisms:** Explore the creation of automated systems that can detect and neutralize attacks in real-time, effectively reducing the window of vulnerability for LLM-powered applications.
- 3) **Integration with DevSecOps:** Investigate strategies for integrating threat modeling and risk analysis practices within the DevSecOps pipeline to enhance the security posture of applications from the outset.
- 4) **Cross-disciplinary Case Studies:** Conduct comprehensive case studies across various industries to validate the applicability of the proposed threat modeling frameworks and to customize defensive strategies accordingly.
- 5) **User Education and Awareness:** Assess the impact of user education on mitigating risks posed by social engineering and similar human-targeted attacks in the context of LLM interactions.

TABLE II
THREAT RANKING FOR LLM-POWERED APPLICATION

Threat	Damage	Reproducibility	Exploitability
Spoofing (STRIDE)	High	High	Medium
Tampering (STRIDE)	High	Medium	High
Repudiation (STRIDE)	Medium	Medium	Low to Medium
Information Disclosure (STRIDE)	High	Medium	Medium
Denial of Service (STRIDE)	High	High	High
Elevation of Privilege (STRIDE)	High	Low	Low

REFERENCES

- [1] "Bing search," <https://www.bing.com/>, 2023.
- [2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, and et al., "Language models are few-shot learners," in *NeurIPS*, 2020.
- [3] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, and A. e. a. Passos, "Palm 2 technical report," *arXiv preprint arXiv:2305.10403*, 2023.
- [4] S. Pichai, "An important next step on our ai journey," <https://blog.google/technology/ai/bard-google-ai-search-updates/>, 2023.
- [5] "Chatgpt plugins," <https://openai.com/blog/chatgpt-plugins>, 2023.
- [6] "Chatwithpdf," <https://gptstore.ai/plugins/chatwithpdf-sdan-io>, 2023.
- [7] "Introducing chatgpt," <https://openai.com/blog/chatgpt>, 2023.
- [8] "Jailbreaking gpt-4's code interpreter," <https://www.lesswrong.com/posts/KSroBnxCHodGmPPJ8/jailbreaking-gpt-4-s-code-interpreter>, 2023.
- [9] "Learn prompting," <https://learnprompting.org/>, 2023.
- [10] "llma2-13b-chat-url," <https://huggingface.co/meta-llama/Llama-2-7b>, 2023.
- [11] "llma2-7b-chat-url," <https://huggingface.co/meta-llama/Llama-2-13b-chat-hf>, 2023.
- [12] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for gpt-3?" in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, 2022.
- [13] S. Shin, S.-W. Lee, H. Ahn, S. Kim, H. Kim, B. Kim, K. Cho, G. Lee, W. Park, J.-W. Ha, and N. Sung, "On the effect of pretraining corpora on in-context learning by a large-scale language model," in *NAACL*, 2022.
- [14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *NeurIPS*, 2022.
- [15] S. Tete, (2024) Llm-doctor: An llm-powered doctor's assistant. Accessed on 15th January 2024. [Online]. Available: <https://github.com/cehstephen/LLM-Doctor>
- [16] K. H. Kim, K. Kim, and H. K. Kim, "Stride-based threat modeling and dread evaluation for the distributed control system in the oil refinery," *ETRI Journal*, vol. 44, no. 6, pp. 991–1003, 2022.
- [17] W. Xiong and R. Lagerström, "Threat modeling – a systematic literature review," *Computers & Security*, vol. 84, pp. 53–69, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404818307478>
- [18] A. Karahasanovic, P. Kleberger, and M. Almgren, "Adapting threat modeling methods for the automotive industry," in *Proceedings of the 15th ESCAR Conference*, 2017, pp. 1–10.
- [19] L. Mauri and E. Damiani, "Modeling threats to ai-ml systems using stride," *Sensors*, vol. 22, no. 17, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/17/6662>
- [20] S. Lipner, "The trustworthy computing security development lifecycle," in *20th Annual Computer Security Applications Conference*, Tucson, AZ, USA, 2004, pp. 2–13.
- [21] T. G. Dietterich, "Steps toward robust artificial intelligence," *AI Magazine*, vol. 38, no. 3, pp. 3–24, 2017. [Online]. Available: <https://doi.org/10.1609/aimag.v38i3.2756>
- [22] N. Mirehghallah, H. Kim, X. Zhou, Y. Tsvetkov, M. Sap, R. Shokri, and Y. Choi, "Can llms keep a secret? testing privacy implications of language models via contextual integrity theory," 2023.
- [23] E. N. Crothers, N. Japkowicz, and H. L. Viktor, "Machine-generated text: A comprehensive survey of threat models and detection methods," *IEEE Access*, vol. 11, pp. 70977–71002, 2023.
- [24] N. Carlini, M. Jagielski, C. A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, and F. Tramèr, "Poisoning web-scale training datasets is practical," *arXiv preprint arXiv:2302.10149*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.10149>
- [25] —, "Poisoning web-scale training datasets is practical," 2023.
- [26] J. Yi, Y. Xie, B. Zhu, K. Hines, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," 2023.
- [27] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Prompt injection attacks and defenses in llm-integrated applications," 2023.
- [28] R. Pedro, D. Castro, P. Carreira, and N. Santos, "From prompt injections to sql injection attacks: How protected is your llm-integrated web application?" 2023.
- [29] D. A. Kindy and A.-S. K. Pathan, "A detailed survey on various aspects of sql injection in web applications: Vulnerabilities, innovative attacks, and remedies," 2013.
- [30] R. Jahanshahi, A. Doupe, and M. Egele, "You shall not pass: Mitigating sql injection attacks on legacy web applications," in *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '20. ACM, Oct. 2020. [Online]. Available: <http://dx.doi.org/10.1145/3320269.3384760>
- [31] H. Li, D. Guo, W. Fan, M. Xu, J. Huang, F. Meng, and Y. Song, "Multi-step jailbreaking privacy attacks on chatgpt," *arXiv preprint arXiv:2304.05197*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.05197>
- [32] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. Anderson, Y. Singer, and A. Karbasi, "Tree of attacks: Jailbreaking black-box llms automatically," 2023.
- [33] Y. Wu, X. Li, Y. Liu, P. Zhou, and L. Sun, "Jailbreaking gpt-4v via self-adversarial attacks with system prompts," 2023.
- [34] Z. Zhang, J. Yang, P. Ke, and M. Huang, "Defending large language models against jailbreaking attacks through goal prioritization," 2023.
- [35] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "Smoothllm: Defending large language models against jailbreaking attacks," 2023.
- [36] S. Jiang, X. Chen, and R. Tang, "Prompt packer: Deceiving llms through compositional instruction with hidden attacks," 2023.
- [37] OWASP. (2023) Owasp top 10 for large language model applications. OWASP Foundation. Accessed: 26 January 2024. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [38] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt injection attack against llm-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023.
- [39] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Prompt injection attacks and defenses in llm-integrated applications," *arXiv preprint arXiv:2310.12815*, 2023.
- [40] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.15043>
- [41] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P. Y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline defenses for adversarial attacks against aligned language models," *arXiv preprint arXiv:2309.00614*, 2023.
- [42] I. Provilkov, D. Emelianenko, and E. Voita, "Bpe-dropout: Simple and effective subword regularization," 2020.
- [43] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *USENIX Security Symposium*, 2024.
- [44] Z. Braiterman, A. Shostack, J. Marcil, S. de Vries, I. Michlin, K. Wuyts, R. Hurlbut, B. S. Schoenfeld, F. Scott, M. Coles, C. Romeo, A. Miller, I. Tarandach, A. Douglan, and M. French. Threat modeling manifesto. Threat Modeling Manifesto. Accessed: 10 January 2024. [Online]. Available: <https://www.threatmodelingmanifesto.org/>
- [45] I. Michlin. (2021, April 7) Using the threat modeling manifesto to get your team going. Publisher: Security Intelligence, Accessed: January 10, 2023. [Online]. Available: <https://securityintelligence.com/posts/using-threat-modeling-manifesto-to-get-going/>

- [46] I. Tarandach. Continuous threat modeling for development teams. OWASP BeNeLux-Day 2020. [Online]. Available: <https://www.youtube.com/watch?v=ARjIRFQN7XM&t=3665>
- [47] A. Shostack, *Threat Modeling: Designing For Security*. John Wiley & Sons, 2014.
- [48] D. B. Johnson. New manifesto offers ciscos an agile guide to threat modeling. SC Magazine. [Online]. Available: <https://www.scmagazine.com/news/threat-hunting/new-manifesto-offers-ciscos-an-agile-guide-to-threat-modeling>
- [49] R. Harang. (2023) Securing llm systems against prompt injection. Accessed: 08 January 2024. [Online]. Available: <https://developer.nvidia.com/blog/securing-llm-systems-against-prompt-injection/>
- [50] OWASP Foundation. OWASP Top 10 for Large Language Model Applications. Accessed: 08 January 2024. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [51] S. Willison. Prompt injection attacks against gpt-3. Accessed: 08 January 2024. [Online]. Available: <https://simonwillison.net/2022/Sep/12/prompt-injection/>
- [52] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS ML Safety Workshop*, 2022.
- [53] H. J. Branch, J. R. Cefalu, J. McHugh, L. Hujer, A. Bahl, D. del Castillo Iglesias, R. Heichman, and R. Darwishi, "Evaluating the susceptibility of pre-trained language models via handcrafted adversarial examples," *arXiv preprint arXiv:2209.02128*, 2022.
- [54] S. Willison. (2023) Delimiters won't save you from prompt injection. [Online]. Available: <https://simonwillison.net/2023/May/11/delimiters-wont-save-you>
- [55] ———. (2022) Prompt injection attacks against gpt-3. [Online]. Available: <https://simonwillison.net/2022/Sep/12/prompt-injection/>